Programación de Servicios Y Procesos

2º DAM

Soraya Peceño Capilla

speccap273@g.educaand.es

Objetivos

- Conceptos: **concurrencia multiprogramación**.
- Entender las **políticas de planificación** del sistema para proporcionar multiprogramación y multitarea.
- Generación de lanzadores desde Eclipse
- Generación de lanzadores desde clases Java
- Información sobre procesos lanzados
- Comunicación entre procesos
  - De errores
  - De mensajes de entrada y salida

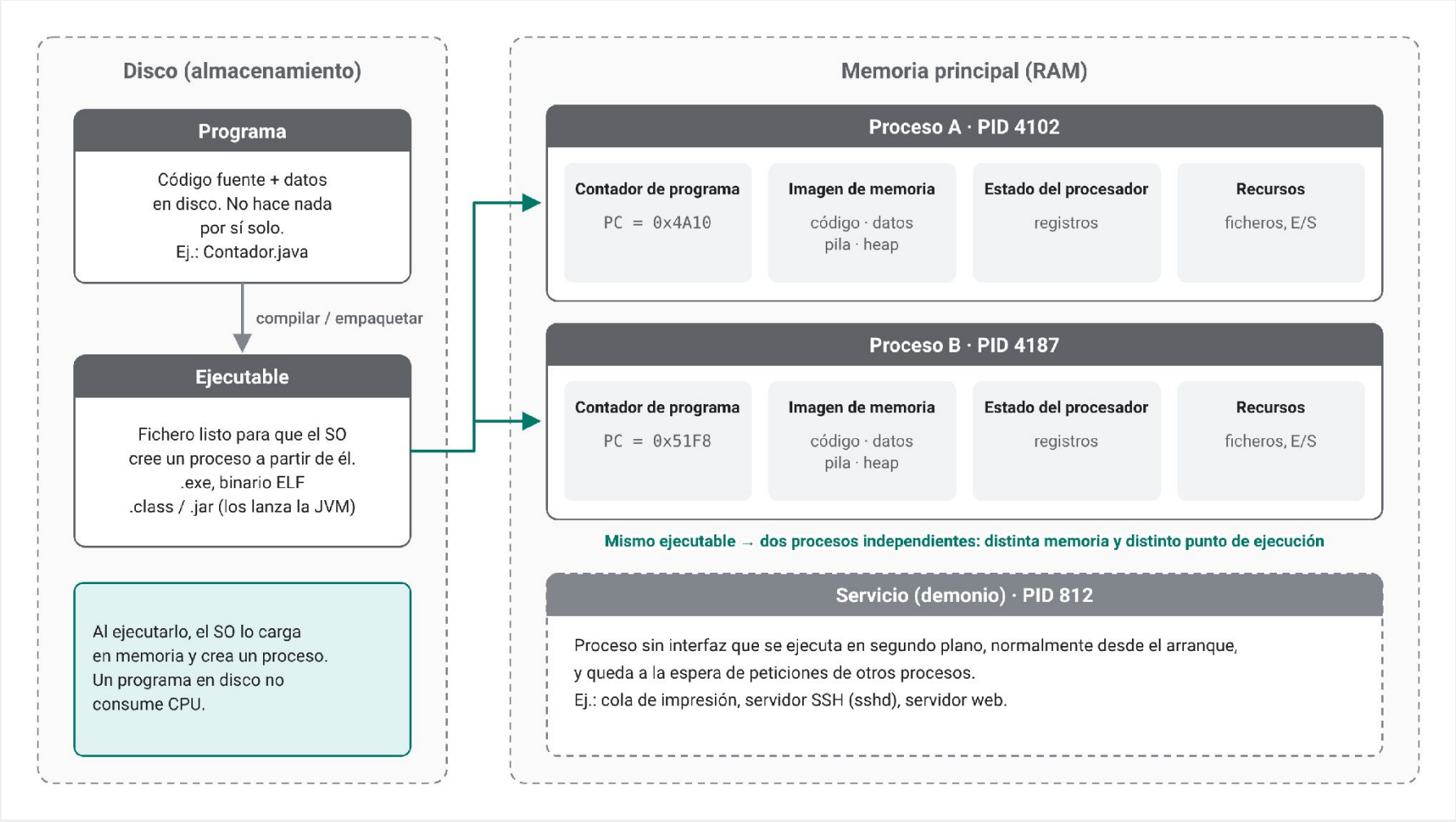
Conceptos

- **Programa**: toda la información (tanto **código como datos**) almacenada en disco de una aplicación, que resuelve una necesidad concreta para los usuarios.
- **Proceso**: **ejecución** de un programa, podremos decir de manera muy simplificada que es un proceso. Y no se refiere únicamente al código y a los datos, sino que incluye todo lo **necesario para su ejecución**:
  - Un **contador** del programa: algo que indique **por dónde se está ejecutando.**
  - Una **imagen de memoria**: es el espacio de memoria que el proceso está utilizando.
  - **Estado del procesador**: se define como el valor de los registros del procesador sobre los cuales se está ejecutando.

Conceptos

- Los **procesos** son **entidades independientes** aunque ejecuten el mismo programa. Pueden coexistir dos procesos que ejecuten el mismo programa, pero con diferentes datos (es decir, con distintas imágenes de memoria) y en distintos momentos de su ejecución (con diferentes contadores de programa).
- **Ejecutable**: un fichero ejecutable contiene la información necesaria para crear un proceso a partir de los datos almacenados de un programa. Es decir, llamaremos "ejecutable" al fichero que permite poner el programa en ejecución como proceso.
- **Demonio (o servicio)**: proceso **no interactivo** que está ejecutándose **continuamente en segundo plano**, sin ninguna intermediación del usuario. Suelen proporcionar un servicio básico para el resto de procesos.

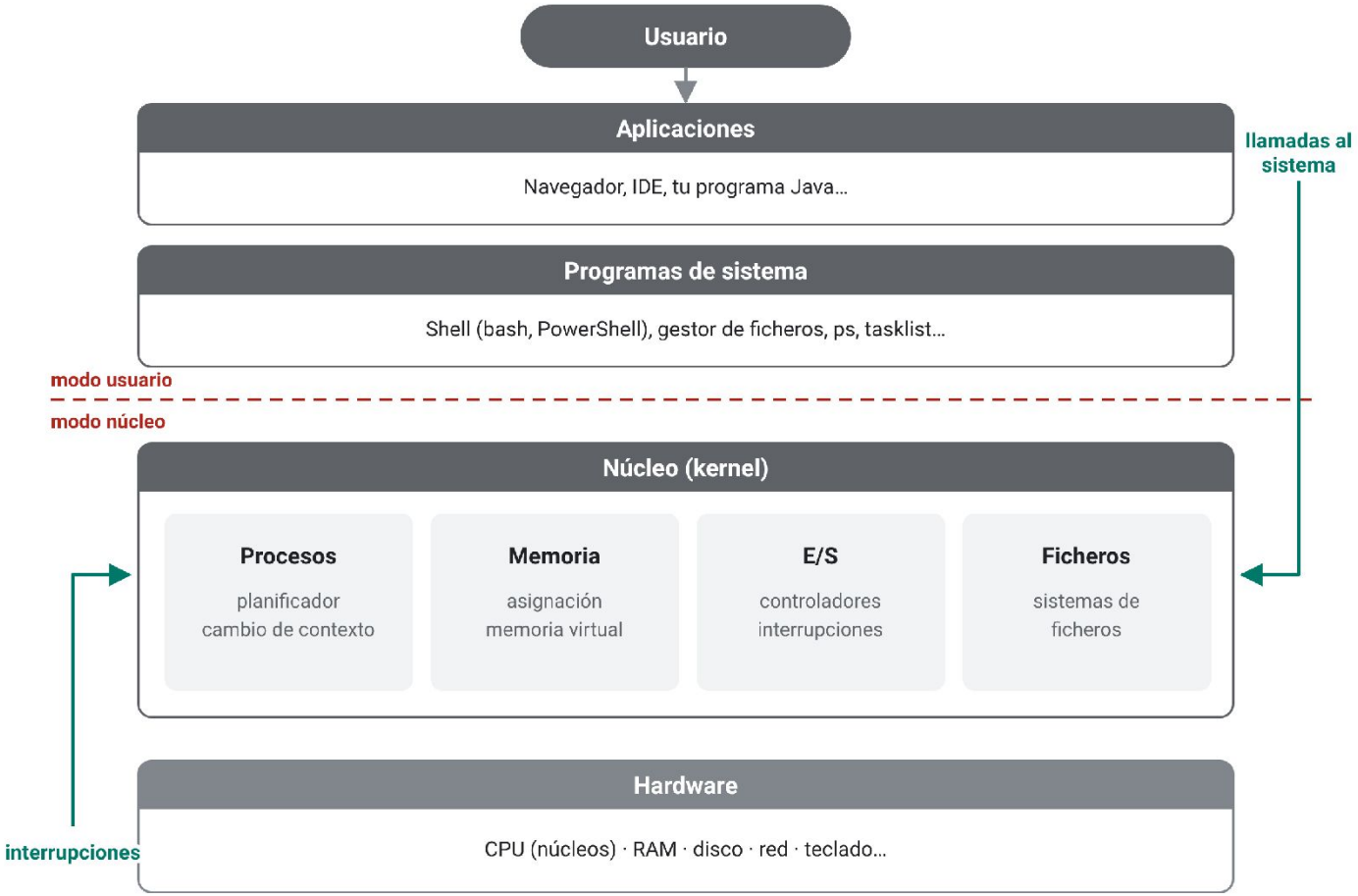
Conceptos


Disco (almacenamiento)
Programa
Código fuente + datos
en disco. No hace nada
por sí solo.
Ej.: Contador.java
compilar / empaquetar
Ejecutable
Fichero listo para que el SO
cree un proceso a partir de él.
.exe, binario ELF
.class / .jar (los lanza la JVM)
Al ejecutarlo, el SO lo carga
en memoria y crea un proceso.
Un programa en disco no
consume CPU.
Memoria principal (RAM)
Proceso A · PID 4102
Contador de programa
PC = 0x4A10
Imagen de memoria
código · datos
pila · heap
Estado del procesador
registros
Recursos
ficheros, E/S
Proceso B · PID 4187
Contador de programa
PC = 0x51F8
Imagen de memoria
código · datos
pila · heap
Estado del procesador
registros
Recursos
ficheros, E/S
Mismo ejecutable → dos procesos independientes: distinta memoria y distinto punto de ejecución
Servicio (demonio) · PID 812
Proceso sin interfaz que se ejecuta en segundo plano, normalmente desde el arranque,
y queda a la espera de peticiones de otros procesos.
Ej.: cola de impresión, servidor SSH (sshd), servidor web.

Conceptos

- **Sistema operativo**: intermediario entre el usuario y las aplicaciones que utiliza y el hardware del ordenador.
  - Ejecutar los programas del usuario.
  - Hace de interfaz entre el usuario y los recursos del ordenador
  - Utiliza los recursos del computador de forma eficiente. Encargado de repartir los recursos en función de sus políticas a aplicar.

Conceptos


Usuario
Aplicaciones
Navegador, IDE, tu programa Java...
llamadas al sistema
Programas de sistema
Shell (bash, PowerShell), gestor de ficheros, ps, tasklist...
modo usuario
modo núcleo
Núcleo (kernel)
Procesos
planificador
cambio de contexto
Memoria
asignación
memoria virtual
E/S
controladores
interrupciones
Ficheros
sistemas de ficheros
Hardware
CPU (núcleos) · RAM · disco · red · teclado...
interrupciones

Programación Concurrente

La computación **concurrente** permite la **posibilidad de tener en ejecución al mismo tiempo múltiples tareas interactivas**. Es decir, permite realizar varias cosas al mismo tiempo, como escuchar música, visualizar la pantalla del ordenador, imprimir documentos, etc. Dichas tareas se pueden ejecutar en 2 contextos diferentes:

- MULTIPROGRAMACIÓN (**Monoprocesador**: Simula varios procesos)
- PROCESAMIENTO PARALELO (**Multiprocesador** : Ejecución real de varios procesos en paralelo)

Programación Concurrente: Multiprogramación

- Se basa en que el SSOO gestiona su **monoprocesador** para que se **simulen** que varios procesos se ejecutan al mismo tiempo. Aunque si sólamente existe un procesador, no es algo real.
- Se van **intercambiando** los procesos. Los procesos van entrando y saliendo de la ejecución (reciben o no tiempo de la CPU)
- La programación concurrente **mejor la experiencia de usuario**, ya que parece que se ejecuten a la misma vez.

  Ejemplo: a la vez estamos modificando un documento de texto, escuchando música por Internet o navegando con el Chrome. Si los cambios se producen suficientemente rápidos, parece que todo se ejecuta al mismo tiempo y sin cortes del servicio

Programación Concurrente

Concurrencia en 1 núcleo (multiprogramación)



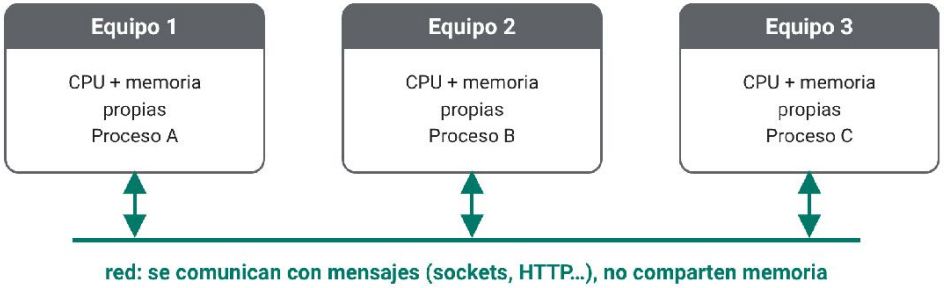
Se alternan: parecen simultáneas, pero en cada instante solo avanza una.

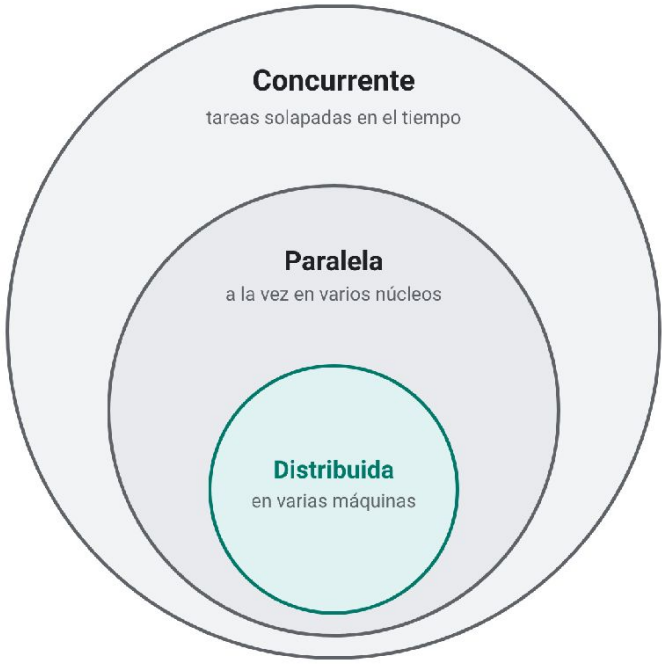
Paralelismo (varios núcleos)



Ejecución simultánea real. Cada núcleo, además, sigue alternando tareas.

Programación distribuida (varias máquinas)





Todo programa paralelo es concurrente;
no al revés.

Programación Concurrente: Procesamiento en paralelo

- Ocurre en sistemas multiprocesador, considerando también multiprocesador los modelos con varios núcleos o cores. Ejemplo Dual Cores, Quad Cores, i3, i5, i7 ...
- Cada **núcleo** podría estar ejecutando **una instrucción de un proceso diferente** al mismo tiempo
- **Procesador lógico**: A su vez cada núcleo puede tener varios planificacores para el SSOO.
  - Sin Hyper-Threading: 1 núcleo = 1 procesador lógico.
  - Con Hyper-Threading (**SMT**): 1 núcleo = 2 procesadores lógicos.

Programación Concurrente: Procesamiento en paralelo

Investiga: ¿Qué CPU tiene tu equipo?

- *Administrador de Tareas*
- *Java System.out.println(Runtime.getRuntime().availableProcessors());*
- *PowerShell:*

  Get-CimInstance Win32_ComputerSystem | Select-Object NumberOfProcessors, NumberOfLogicalProcessors

  Get-CimInstance Win32_Processor | Select-Object Name, NumberOfCores, NumberOfLogicalProcessors

a) ¿Tu equipo es multiprocesador, multinúcleo o las dos cosas? Justifícalo con los datos.

b) ¿Por qué hay más procesadores lógicos que núcleos? ¿Qué tecnología lo permite?

Programación Concurrente: Procesamiento en paralelo

- Los sistemas **multiprocesadores** se pueden clasificar dependiendo de su arquitectura en:
  - Sistemas multiprocesador **débilmente acoplados.** Usan **memoria distribuida** (no comparten memoria) pudiendo trabajar de manera independiente en paralelo.

    La comunicación entre procesos se realiza a través de mensajes, en lugar de acceder directamente a una memoria compartida.

    Un ejemplo sería el caso en el que cada procesador está físicamente separado del otro. Una red de ordenadores o el mismo Internet en un ejemplo de sistema débilmente acoplado

Programación Concurrente: Procesamiento en paralelo

- Sistemas multiprocesador **fuertemente acoplados.** Los procesadores comparten memoria y se conectan a ella por un bus de datos. También pueden compartir una caché de datos.

  Existe fuerte acoplación ya que comparten variables en la memoria común.

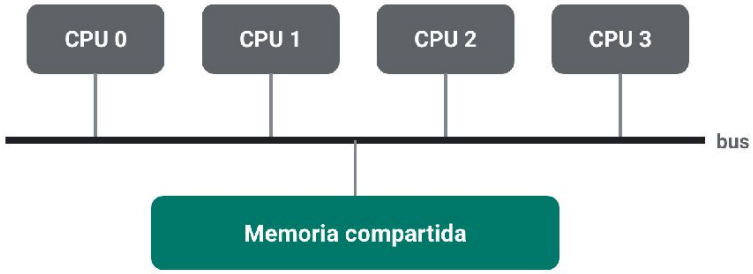
  A su vez pueden clasificarse en:

  - Sistemas multiprocesos **simétricos,** si los procesadores del sistema son similares en cuanto a características y se compite entre iguales.
  - Sistemas multiproceso **asimétricos,** en los que uno de los procesadores es el master y es el que controla y orquesta al resto. Este sistema también se llama **master - slave**

Programación Concurrente: Procesamiento en paralelo

Fuertemente acoplados

memoria compartida

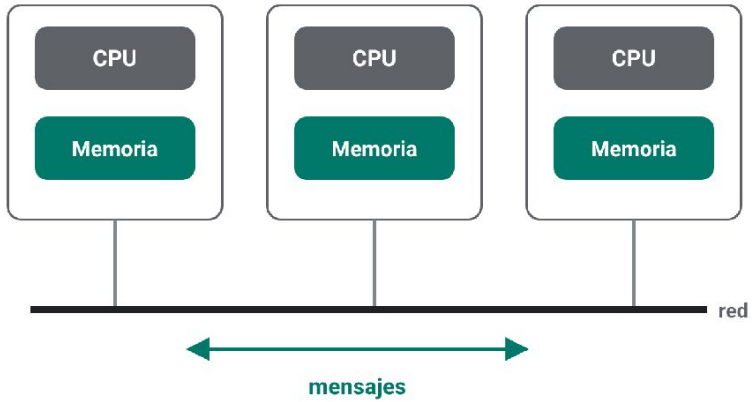


Simétrico (SMP)	Asimétrico
Todas las CPU son iguales. Cualquiera ejecuta cualquier tarea.	Una CPU maestra reparte el trabajo a las demás (maestro-esclavo).

Se comunican leyendo y escribiendo variables en la memoria común → hay que sincronizar.
Ej.: tu portátil o el móvil (multinúcleo).

Débilmente acoplados

memoria distribuida



Cada nodo tiene su propia memoria: no hay variables compartidas; la comunicación es por mensajes.
Ej.: un clúster, una red de equipos, Internet.

Programación Concurrente: Programación en paralelo

- La **programación en paralelo** ocurre con procesadores **multinúcleos**. En ellos, el **SSOO planifica** qué trabajos se ejecutan en cada núcleo pudiendo ocurrir cambio para generar multitarea.
- Antes un procesador con varios cores: todos los cores comparten la misma memoria, por los que es necesario esa coordinación. A esto se le llama programación en paralelo.
- Cada núcleo puede ejecutar diferentes procesos o tareas de un mismo proceso (diferentes hilos de ejecución)
- La programación paralela se puede utilizar conjuntamente con la programación concurrente, permitiendo al mismo tiempo multiprogramación.

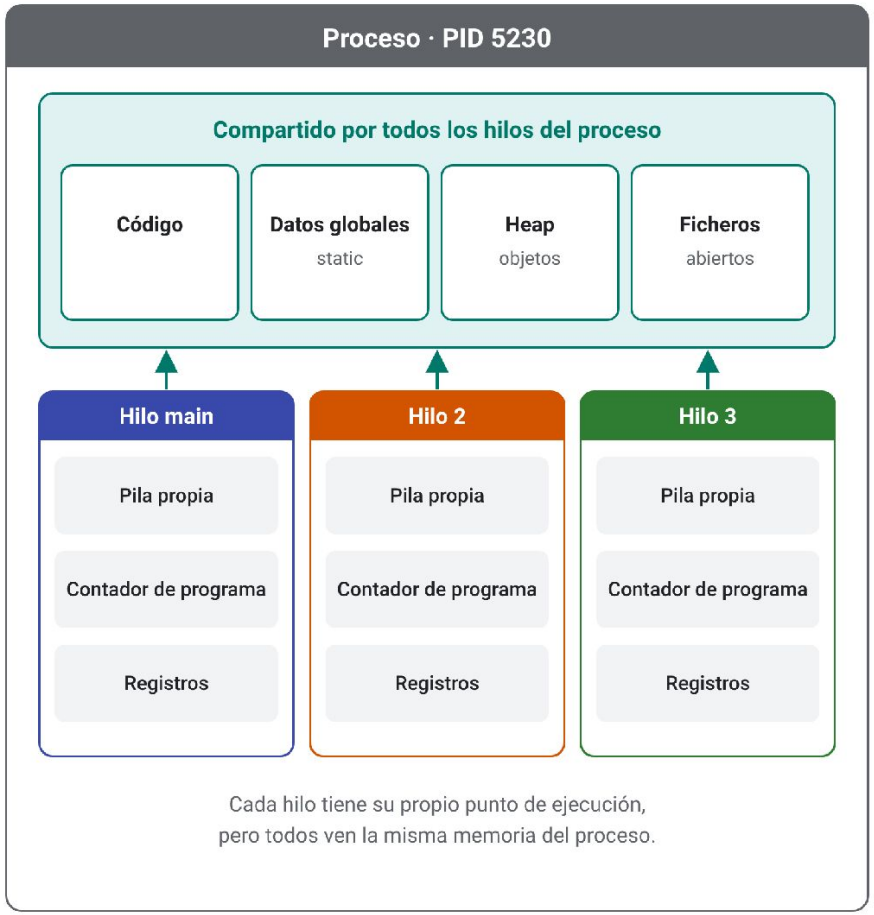
Programación Concurrente: Hilo

Un hilo o hilo de ejecución,: unidad más pequeña de procesamiento que puede ser gestionada de manera independiente por un sistema operativo. En términos más simples, es una secuencia de instrucciones que el procesador puede ejecutar.

Los hilos permiten que un programa realice múltiples tareas al mismo tiempo, lo que se conoce como **multitarea**. Por ejemplo, en una aplicación de procesamiento de texto, un hilo podría estar encargado de la interfaz de usuario, mientras que otro hilo podría estar guardando el documento en segundo plano. Esto mejora la **eficiencia y la capacidad de respuesta**

Los hilos **comparten el mismo espacio de memoria** dentro de un proceso, lo que significa que pueden acceder a los **mismos datos y recursos**, pero también implica que deben manejar cuidadosamente el acceso a esos recursos para evitar **conflictos**.

Programación Concurrente: Hilo



	Proceso	Hilo
Memoria	propia y aislada	compartida con el proceso
Crear / cambiar	costoso	ligero
Comunicación	IPC: tuberías, sockets	variables compartidas
Riesgo	si falla, no afecta a los demás	condiciones de carrera: hay que sincronizar
En Java	ProcessBuilder	Thread, ExecutorService

Un proceso tiene al menos un hilo (main).
Proceso = unidad pesada de ejecución;
hilo = unidad ligera.

Programación Concurrente: Programación Distribuida

Varios ordenadores/servidores distribuidos en red.

Cada uno de ellos tendrá sus **propios procesadores**, su propio SSOOy su propia **memoria**.

La programación distribuida posibilita la utilización de un gran número de dispositivos (ordenadores) de forma paralela, lo que permite alcanzar elevadas mejoras en el rendimiento de la ejecución de programas distribuidos.

Sin embargo, como cada ordenador posee su propia memoria, imposibilita que los procesos puedan comunicarse compartiendo memoria, teniendo que utilizar otros esquemas de comunicación más complejos.

Sistema Operativo

Arquitectura o capas del SSOO:

- **Kernel** o núcleo . Es el responsable de gestionar los recursos del ordenador, permitiendo su uso a través de llamadas al sistema.
- Resto de programas: programas de sistema.

Sistema Operativo



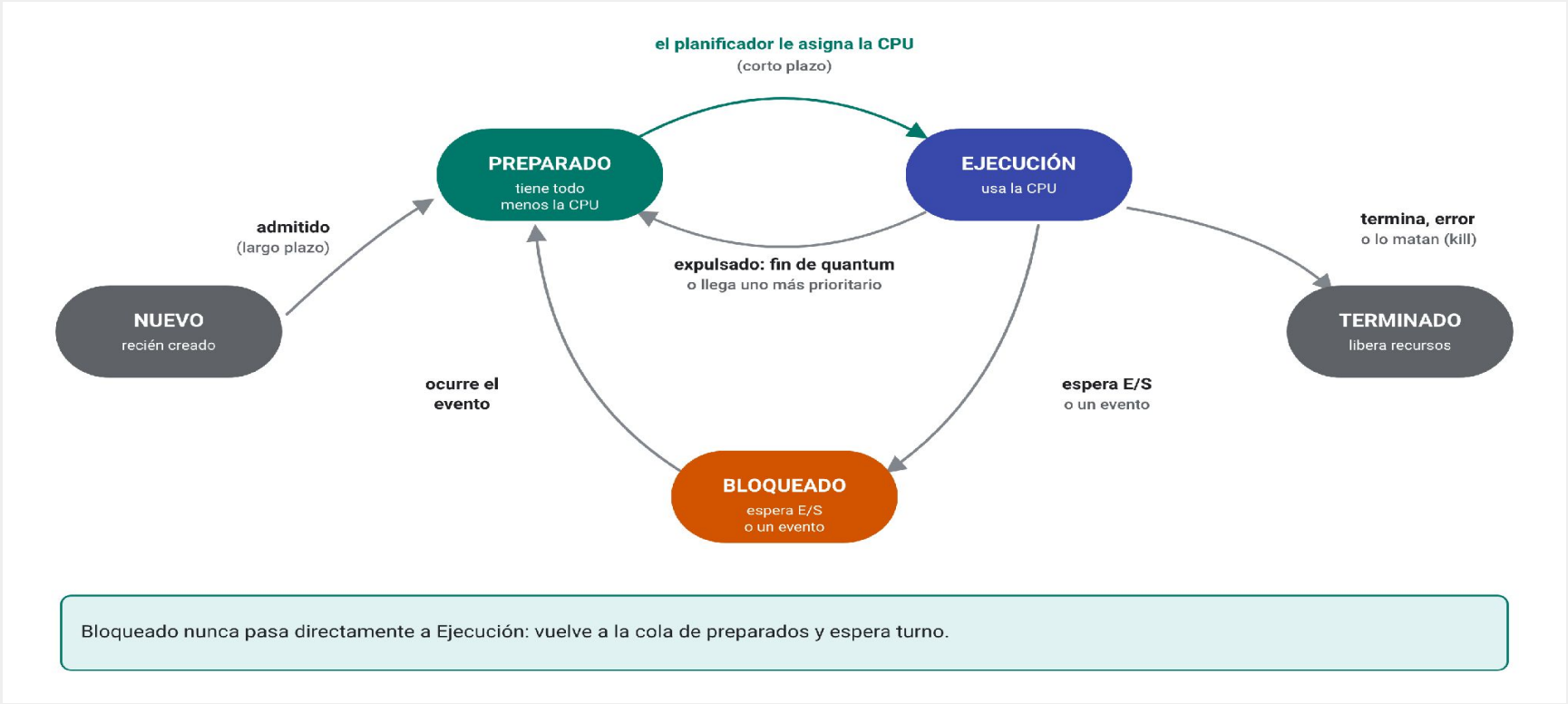
1. La CPU termina la instrucción en curso y salta al núcleo.
2. El núcleo guarda el contexto del proceso (PC, registros) en su PCB.
3. Ejecuta la rutina de servicio de la interrupción, que es código del núcleo.
4. El planificador decide quién sigue: el mismo proceso u otro (cambio de contexto).

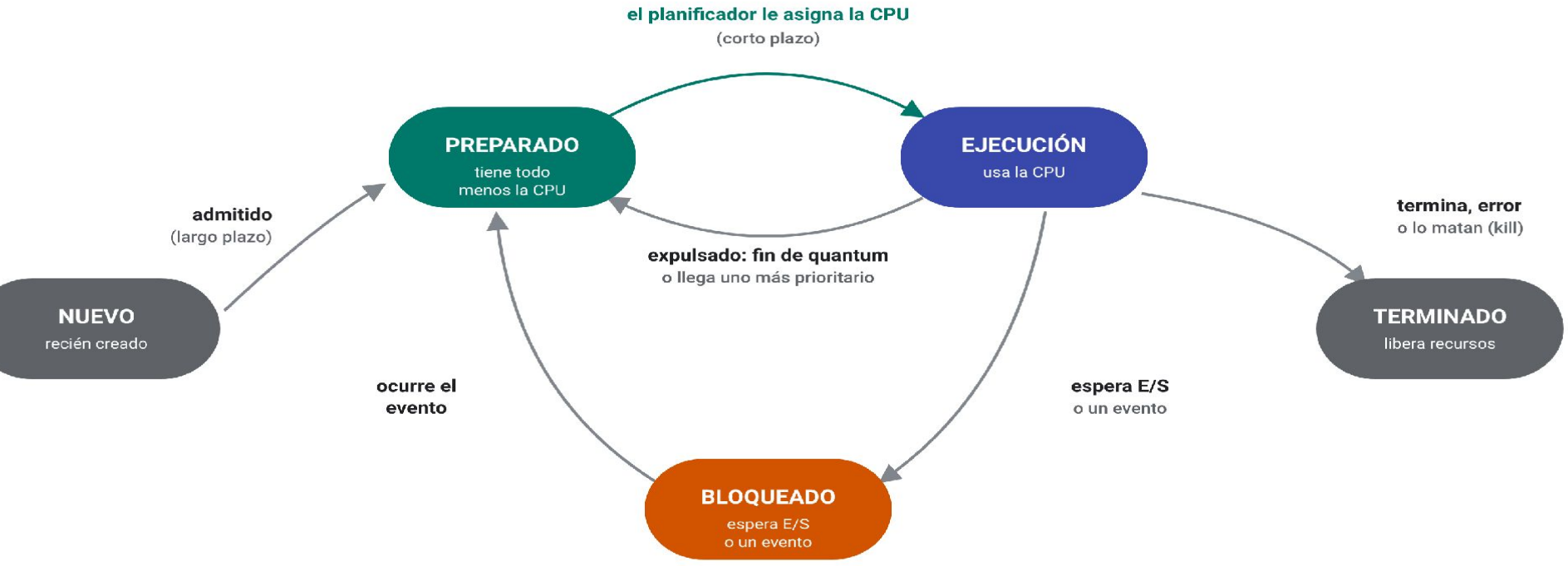
Prioridades: una interrupción más prioritaria puede interrumpir a una rutina en curso;
las de menor prioridad se enmascaran (quedan pendientes) hasta que termine.

Procesos

El sistema operativo es el encargado de poner en ejecución y gestionar los procesos. A lo largo de su ciclo de vida, los procesos pueden cambiar de estado.

Estados de los procesos:





Bloqueado nunca pasa directamente a Ejecución: vuelve a la cola de preparados y espera turno.

Procesos

NUEVO: Recién creado de un fichero ejecutable.

PREPARADO: a la espera de poder ejecutarse. Tiene todo menos la CPU.

EN EJECUCIÓN: El SSOO utiliza el mecanismo de interrupciones para controlar su ejecución. Si el proceso necesita un recurso, incluyendo la realización de operaciones de entrada/salida (E/S), llamará a la llamada del sistema correspondiente. Si un proceso en ejecución se ejecuta durante el tiempo máximo permitido por la política del sistema, salta un temporizador que lanza una interrupción. En este último caso, si el sistema es de tiempo compartido, lo para y lo pasa al estado

Procesos

BLOQUEADO: el proceso está bloqueado esperando que ocurra algún suceso (esperando por una operación de E/S, bloqueado para sincronizarse con otros procesos, etc.). Cuando ocurre el evento que lo desbloquea, el proceso no pasa directamente a ejecución sino que tiene que ser planificado de nuevo por el sistema, es decir, pasa a PREPARADO.

TERMINADO: el proceso ha finalizado su ejecución y libera su imagen de memoria. El proceso termina por dos razones:

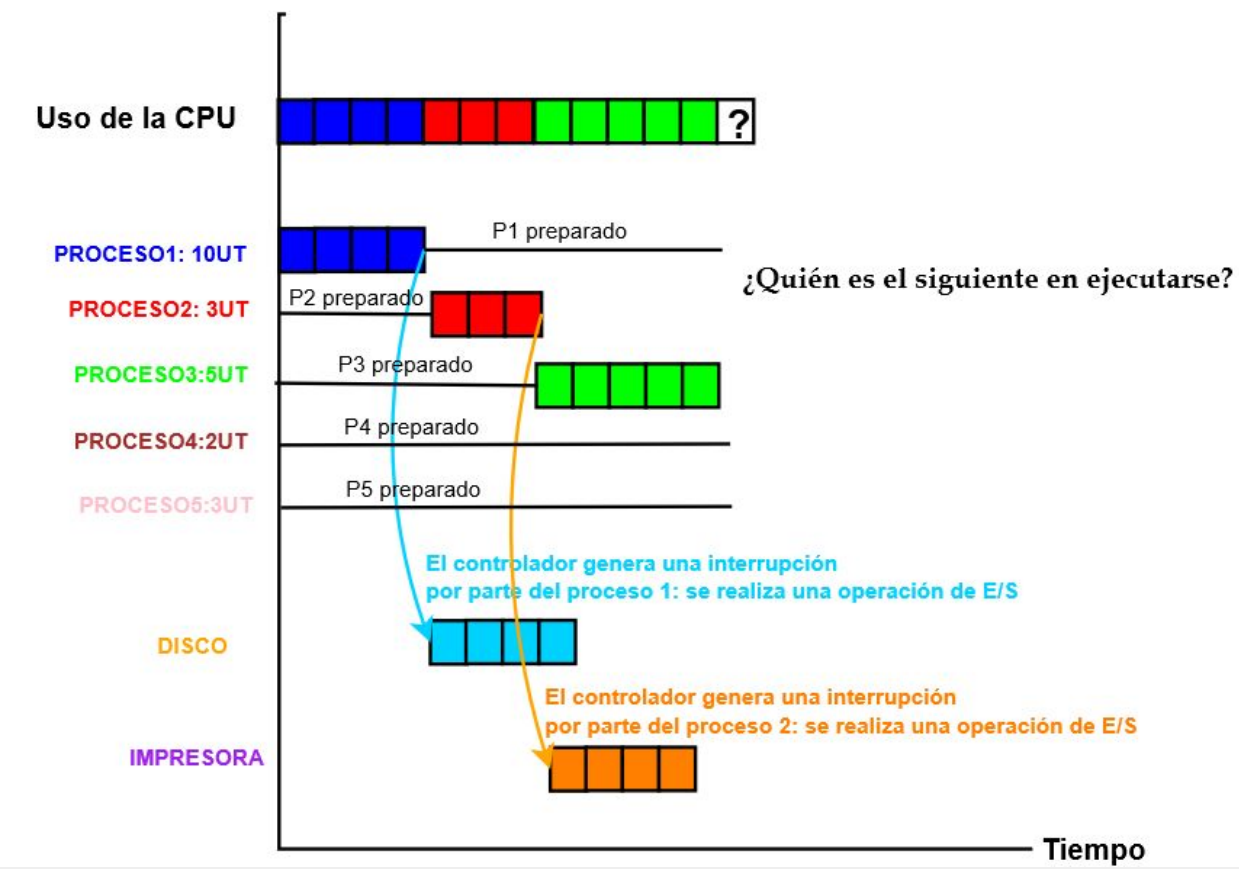
- el propio proceso indica el fin de proceso
- o porque ocurra una excepción

Procesos: Colas de procesos

En sistemas con multiprogramación, el SSOO admite varios procesos en memoria para optimizar el uso de la CPU. Los procesos se irán intercambiando de forma concurrente. Para ello el SSOO organiza los procesos en varias colas:

- Cola de procesos con todos los procesos del sistema
- Cola de preparados, los procesos que están listos para ejecución.
- Cola de dispositivo, los procesos que están a la espera de alguna operación de E/S (procesos bloqueados)

Procesos: Actividad



5 procesos en la cola de procesos. p1 en ejecución pero petición de E/S al disco.

En ese momento, el P2 pasa a ejecución, pero hace otra operación de E/S a la impresora.

Y el P3 pasa a ejecutarse.

Procesos: Criterios de planificación

- **Uso de CPU**: % de tiempo que la CPU está ocupada.
- **Rendimiento:** nº de procesos, trabajos, que se completan por unidad de tiempo.
- **Tiempo de retorno**: tiempo transcurrido entre la llegada de un proceso y su finalización.
- **Tiempo de espera**: tiempo que un proceso permanece en la cola de preparados.
- **Tiempo de respuesta**: tiempo desde que llega la petición hasta la primera respuesta

Procesos: Criterios de planificación

Posibles objetivos de la planificación:

- Minimizar el tiempo medio de espera o de retorno.
- Maximizar la utilización de CPU.
- Mantener el tiempo de respuesta por debajo de un valor máximo.

No existe ninguna política de planificación óptima, depende del objetivo u objetivos a optimizar.

Planificación de procesos

¿Qué proceso pasa a ejecutarse? Depende de la política de ejecución.

- **SJF (Shortest-Job-First)** El proceso más corto o al que le queda menos por ejecutarse → P4
- Si fuera **FIFO** (FCFS, primero en entrar, primero en salir/servir), sería el proceso más antiguo → P1
- **Round Robin** En este caso se le asigna CPU a todos los procesos de manera equitativa
- También podemos establecer como criterio la **prioridad** del proceso.

Veremos todo esto a continuación.

Planificación de procesos

FCFS FIrst Come First Served o FIFO First In First Out

Planificación de servicio por orden de llegada.

Implementación con una cola FIFO

Tiempo espera medio grande

Ejercicio: Calcula:

Calcular :

- tiempo de retorno
- tiempo medio de espera

Proceso	Duración
P1	9
P2	4
P3	2

Planificación de procesos

FCFS FIrst Come First Served o FIFO First In First Out

Proceso	Duración
P1	9
P2	4
P3	2



Tiempo espera: P1=0 P2=9 P3=13

Tiempo de espera medio 22/3

Tiempo de retorno P1= 0 p2=13 P3=15

¿Y si el orden de llegada es: P2, P3, P1?

Planificación de procesos

SJF (Shortest-Job-First): La palabra shortest (el más corto) se refiere al proceso que tenga el el próximo ciclo de CPU más corto de entre todos los procesos PREPARADOS.

El algoritmo SJF se puede comportar de dos formas:

- **Sin expropiación**: Cuando un proceso toma la CPU, ningún otro proceso podrá apropiarse de ella hasta que el proceso que la posee termine de ejecutarse.
- **Con expropiación (también llamado SRTF):** Si se incorpora un nuevo proceso a la cola de PREPARADOS y este tiene un ciclo de CPU **menor** que el ciclo de CPU del proceso que se está ejecutando, entonces dicho proceso es **desalojado** y el nuevo proceso toma la CPU. El que estaba ejecutándose hasta ese momento actualiza el número de unidades de tiempo que le restan.

Planificación de procesos

SJF (Shortest-Job-First)

Calcular el tiempo medio de espera que resulta de aplicar:

- Un algoritmo SJF no expropiativo
- Un algoritmo SJF con expropiación: SRTF

Proceso	Llegada	Duración
P1	0	7
P2	2	4
P3	4	1
P4	5	4

Planificación de procesos

RR (Round Robin):

- Adecuado para implementar tiempo compartido.
- Comportamiento como FCFS, con la diferencia de que cada proceso dispone de un **cuanto (quantum) de tiempo máximo.**
- Si cuando expira el cuanto de tiempo el proceso continúa en CPU, el planificador lo desaloja y lo ingresa al final de la cola de preparados.
- La cola de preparados se gestiona como **FIFO**.
- Si Q es muy grande, los procesos terminan sus ráfagas de CPU antes de que termine el cuanto: equivalente a FCFS.
- Si Q es muy pequeño se suceden constantemente los cambios de contexto y del rendimiento disminuye mucho.

Planificación de procesos

Datos

Proceso	Llegada	Ráfaga
P1	0	7
P2	2	4
P3	4	1
P4	5	4

Mismos 4 procesos,
4 políticas distintas.

Espera = retorno − ráfaga
Retorno = fin − llegada

SRTF consigue la menor espera media, pero necesita conocer las ráfagas y puede dejar sin CPU a los largos.
RR: peor media, más equidad.

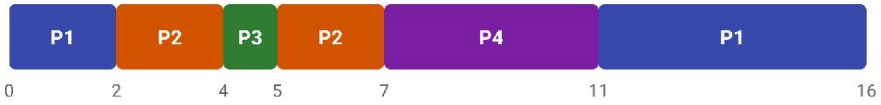
FCFS



SJF (sin expropiación)



SRTF (SJF con expropiación)



Round Robin (q = 2)



Espera media

4,75

4

3

5

Planificación de procesos

Planificación por prioridad:

- Cada proceso tiene una prioridad, entrará primero en la CPU el que tenga mayor prioridad.
- Política de prioridades expulsiva o no.La prioridad se puede definir:
  - De forma interna, la define el SO.
  - De forma externa, la definen los usuarios.
- Inconveniente: Los procesos de prioridad más baja tienen riesgo de inanición.
- Solución: envejecimiento. Ir aumentando de forma progresiva la prioridad de los procesos en espera.

Planificación de procesos

En la **planificación** (scheduling) de procesos se busca conciliar los siguientes objetivos:

- **Equidad**. Todos los procesos deben poder ejecutarse.
- **Eficacia**. Mantener ocupada la **CPU un 100 %** del tiempo.
- **Tiempo de respuesta**. Minimizar el tiempo de respuesta al usuario.
- **Tiempo de regreso**. Minimizar el tiempo que deben esperar los usuarios de **procesos por lotes** para obtener sus resultados. (Procesos dependientes)
- **Rendimiento**. Maximizar el número de tareas procesadas por hora.

Planificación de procesos

- La política de planificación utilizada por **Windows es Round Robin entre procesos de la misma prioridad**
- El sistema operativo **iOS utiliza colas de prioridad** que son un sistema donde los datos tienen un orden de prioridad y se procesan dependiendo de ello. Cuando dos o más procesos tienen la misma prioridad se ejecutan conforme vayan llegando.

Investiga cómo ocurre la planificación en sistemas LINUX (Prioridad dinámica y estática)

Planificación de procesos

- Linux

Clase	Tipo	Cómo reparte la CPU
`SCHED_OTHER`	Normal (la de casi todos los procesos)	Reparto **justo**: el que menos CPU ha usado es el siguiente en entrar. El valor *nice* (de −20 a 19) da más o menos parte: cuanto más bajo, más CPU recibe.
`SCHED_FIFO`	Tiempo real	Por prioridad (1–99). **Sin quantum**: se ejecuta hasta que termina, se bloquea o llega uno más prioritario.
`SCHED_RR`	Tiempo real	Igual que FIFO, pero con **quantum**: los procesos de la misma prioridad se turnan (Round Robin).

Servicios e Hilos

Hemos dicho que todo lo que ejecuta en un equipo es un programa y que cuando está en ejecución se llama proceso.

¿Qué es un servicio? ¿Qué son los hilos?

SERVICIO: es un proceso que normalmente es cargado durante el **arranque del sistema operativo**. Recibe el nombre de servicio, ya que es un proceso que queda **a la espera de que otro le pida** que realice una tarea. Por ejemplo, tenemos el servicio de impresión con su típica cola de trabajos a imprimir.

Para entender el concepto de un **hilo** vamos a pensar en un juego de ordenador con varios personajes. El juego en sí es una aplicación, nosotros controlamos uno de los personajes, los malos también se mueven, interactúan por el escenario y quitan vidas. Cada personaje estará realizando tareas independientes que pueden interactuar entre sí

Servicios e Hilos

HILOS:

Cada uno de los personajes del juego es controlado por un hilo. Todos los hilos forman parte de la misma aplicación, cada uno actúa siguiendo un patrón de comportamiento.

Todos esos hilos comparten la información de la aplicación: el número de vidas restantes, la puntuación obtenida, la posición en la que se encuentra el personaje del usuario y el resto de personajes, etc.

Como sabemos, esas informaciones son variables. Pues bien, **un proceso, no puede acceder directamente a la información de otro proceso**. Pero, **los hilos de un mismo proceso están dentro de él**, por lo que **comparten la información de las variables de ese proceso.**

Servicios e Hilos

HILOS:

Un proceso al menos tiene un hilo de ejecución: el hilo principal.

Un proceso es la unidad pesada de ejecución ya que si es multihilo, podemos considerar a cada hilo como la unidad de ejecución ligera.

Servicios e Hilos

Hyper-Threading (HT):

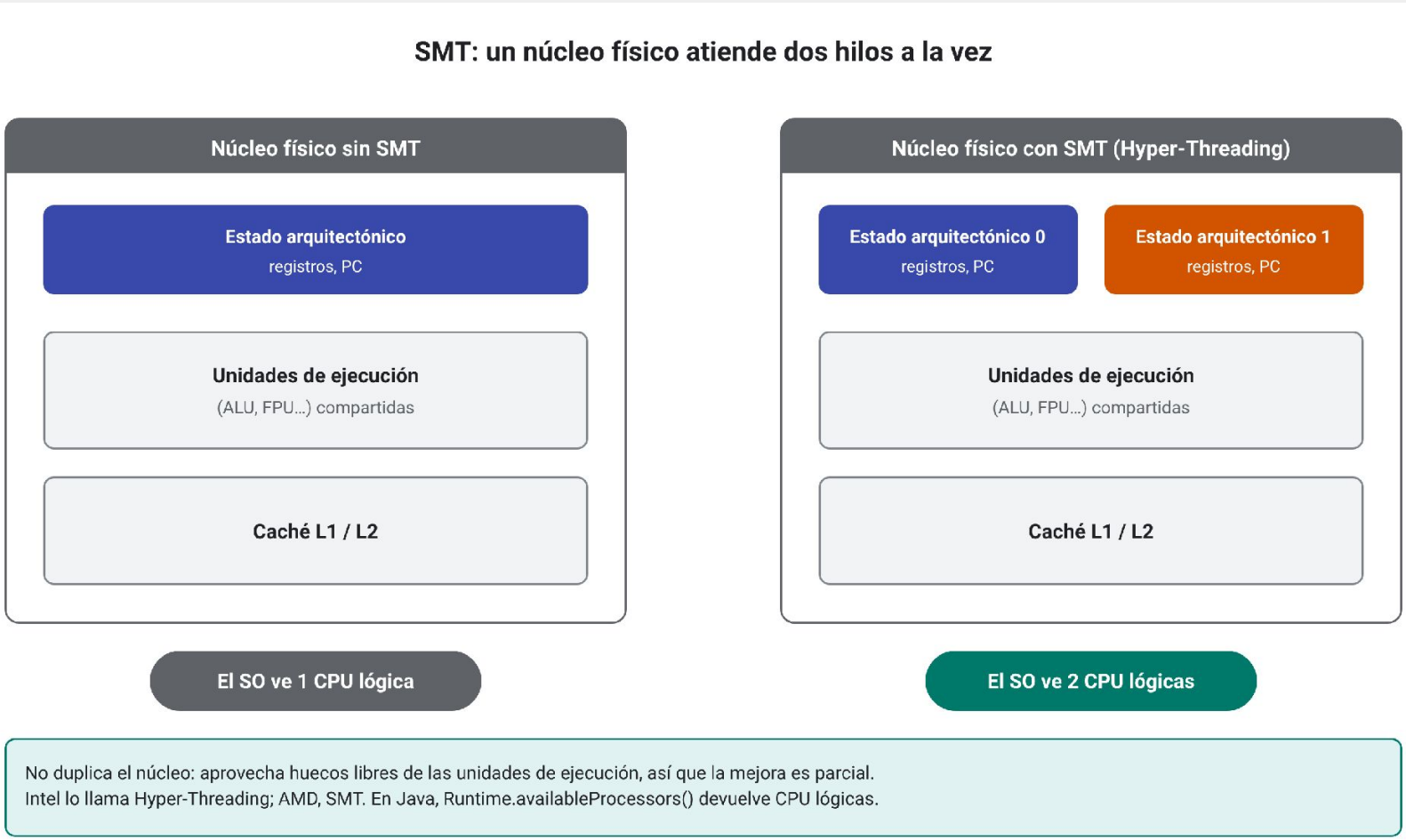
Es una tecnología patentada por Intel, que incorporó en sus micros Pentium4 de un sólo núcleo para que el propio micro (hardware) simulara la existencia de 2 núcleos lógicos, para obtener mayor productividad en procesos de más de un hilo, ya que cada núcleo lógico gestionará cada hilo de forma casi independiente.

Esta tecnología la eliminó en sus Core 2 Duo y Quad; ya que al existir más de un núcleo hardware no hacía falta simular la existencia de más de un núcleo por núcleo físico.

Y lo ha vuelto a introducir en su familia de microprocesadores i7, i5 e i3. Estos últimos, por cada núcleo físico, simulan 2 núcleos lógicos. Buscan así incrementar la productividad del micro.

Hyperthreading-core-i3-y-core-i5

Servicios e Hilos



Árbol de procesos

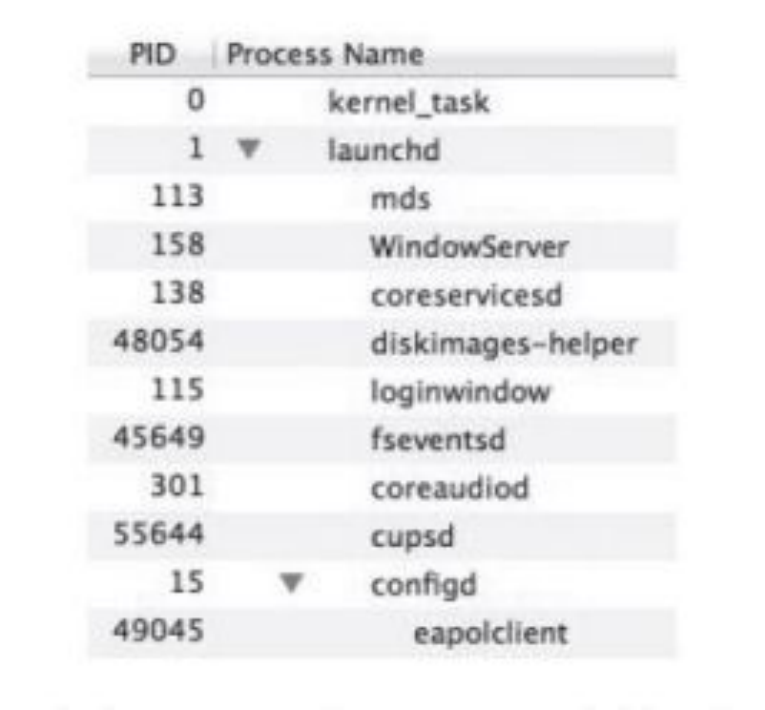
Un proceso es invocado por otro proceso. Esto genera dependencia entre los procesos.

A su vez, el nuevo proceso puede crear nuevos procesos, formándose lo que se denomina un **árbol de procesos.**

Ejemplo: Cuando se arranca el ordenador, y se carga en memoria el **kernel del sistema** a partir de su imagen en disco, se crea el proceso inicial del sistema. A partir de este proceso,se crea el resto de procesos de forma jerárquica, estableciendo padres, hijos, abuelos, etc.

Para identificar a los procesos, los sistemas operativos suelen utilizar un **identificador de proceso (process identifier [PID])** unívoco para cada proceso. Es la forma que tiene el sistema de referirse a los procesos que gestiona.

Árbol de procesos



PID	Process Name
0	kernel_task
1	launchd
113	mds
158	WindowServer
138	coreservicesd
48054	diskimages-helper
115	loginwindow
45649	fseventsd
301	coreaudiod
55644	cupsd
15	configd
49045	eapolclient

Árbol de procesos de Mac OS X

Gestión de procesos: herramientas y comandos

En Windows: Administrador de tareas

En Linux: Monitor de sistemas

Ambos ofrecen, al menos, la siguiente funcionalidad / información:

- Listado de todos los procesos que se encuentran activos en el sistema, mostrando su PID, usuario y ubicación de su fichero ejecutable.
- Posibilidad de finalizar procesos.
- Información sobre el uso de CPU, memoria principal y virtual, red.
- Posibilidad de cambiar la prioridad de ejecución de los procesos.

Gestión de procesos: herramientas y comandos

Investiga y profundiza sobre los siguientes comandos

En windows:

- tasklist
- taskkill

En Linux:

- ps
- pstree
- kill
- killall
- nice

Procesos en Java

La clase abstracta java.lang.Process proporciona métodos para conocer información y gestionar procesos.

Existen otras dos clases: ProcessBuilder y Runtime que nos ayudan a crear y gestionar procesos.

Mecanismo	Clase	Método
Ejecución	Runtime	exec()
Ejecución	ProcessBuilder	start()
Espera	Process	waitFor()
Generación de código de terminación	System	exit(valor_del_retorno)
Obtención de código de terminación	Process	waitFor()

Procesos en Java

Objetivos sobre Procesos:

- Poder arrancar un proceso y hacerle llegar los parámetros de ejecución.
- Poder quedar a la espera de que el proceso termine.
- Poder recoger el código de finalización de ejecución para determinar si el proceso se ha ejecutado correctamente o no.
- Poder leer los datos generados por el proceso para su tratamiento.

Procesos en Java

Clase Runtime:

Nos permite ejecutar comandos del sistema operativo y controlar la máquina virtual de Java (memoria libre, recolector de basura, tiempo de ejecución, etc.)

La ejecución de procesos es responsabilidad del SSOO.

Para asegurar la portabilidad de los programas es aconsejable evitar, en la medida de lo posible, el uso de características específicas del SSOO

Procesos en Java

Clase Runtime:

Toda aplicación Java tiene una **única instancia de la clase Runtime (patrón Singlenton).** Esta permite que la aplicación interactúe con su entorno de ejecución a través del método estático getRuntime.

```
Runtime run = Runtime.getRuntime();
```

El método **getRuntime** constituye un canal de comunicación entre la aplicación y su entorno, posibilitando la interacción con el SSOO (paso a ejecución) a través del método **exec**

Process	exec(String command)
Process	exec(String[] cmdarray)
Process	exec(String[] cmdarray, String[] envp)
Process	exec(String[] cmdarray, String[] envp, File dir)

Procesos en Java

Clase Runtime:

```
public static void main(String[] args)
{
        Runtime rt = Runtime.getRuntime();
        System.out.println("Memoria Libre:" + rt.freeMemory());
        System.out.println("Total Memoria:" + rt.totalMemory());
        System.out.println("Max Memoria:" + rt.maxMemory());
        System.out.println("Nº Procesadores" + rt.availableProcessors());
        System.out.println("JRE Version:" + rt.version());
        System.out.println("Terminando programa...");
        rt.exit(1); //Termina la ejecución de la máquina virtual
}
```

Procesos en Java

Clase Process:

Es una clase abstracta. Representa al proceso de ejecución y permite obtener información sobre este.

Método	Descripción
destroy()	Destruye el proceso sobre el que se ejecuta.
exitValue()	Devuelve el valor de retorno del proceso cuando este finaliza. Sirve para controlar el estado de la ejecución.
getErrorStream()	Proporciona un InputStream conectado a la salida de error del proceso.
getInputStream()	Proporciona un InputStream conectado a la salida normal del proceso.
getOutputStream()	Proporciona un OutputStream conectado a la entrada normal del proceso.
isAlive()	Determina si el proceso está o no en ejecución.
waitFor()	Detiene la ejecución del programa que lanza el proceso a la espera de que este último termine.

Procesos en Java

Este código muestra cómo ejecutar un proceso del sistema:

```
public static void main(String[] args){
        Runtime rt = Runtime.getRuntime();
        String[] informacionProceso = {"notePad.exe"};
        Process proceso;
        try {
            proceso = rt.exec(informacionProceso);
            int codigoRetorno = proceso.waitFor(); //Espero a que termine
            logger.debug(codigoRetorno);
        } catch (IOException e) {
            logger.error(e.getMessage());
        } catch (InterruptedException e) {
            logger.error(e.getMessage());
        }
    }
```

Procesos en Java

1. Modifica el código anterior para que se ejecute el notepad y cree un fichero llamado miFichero.txt
2. ¿Qué pasa cuando ejecutas el programa por segunda vez?
3. ¿Dónde lo crea?
4. Ahora crea una carpeta en c llamada psp y modifica el comando anterior para que lo cree en dicha carpeta
5. Depura el código para saber qué valor tiene de retorno: `proceso.waitFor();`

Ejercicio 1 , 2 y 3 del Boletín 1

Procesos en Java

Clase ProcessBuilder:

La clase ProcessBuilder permite, como Runtime, crear procesos.

Método	Descripción
start	Inicia un nuevo proceso usando los atributos especificados.
command	Permite obtener o asignar el programa y los argumentos de la instancia de ProcessBuilder.
directory	Permite obtener o asignar el directorio de trabajo del proceso.
environment	Proporciona información sobre el entorno de ejecución del proceso.
redirectError	Permite determinar el destino de la salida de errores.
redirectInput	Permite determinar el origen de la entrada estándar.
redirectOutput	Permite determinar el destino de la salida estándar.

ProcessBuilder

Clase ProcessBuilder:

La clase ProcessBuilder permite, como Runtime, crear procesos.

A través de la llamada al **método start se crea un nuevo proceso** en el sistema con los atributos definidos en la instancia de **ProcessBuilder**.

```
String[] comando = {"C:\\Program Files\\Google\\Chrome\\Application\\chrome.exe",
        "https://www.marca.com/"};
ProcessBuilder pb = new ProcessBuilder(comando);
try {
        Process p = pb.start();
} catch (IOException e) {          System.out.println(e);
}
```

Si llamamos varias veces al método start, se crearán tantos nuevos procesos como llamadas hagamos, todos ellos con los mismos atributos. Cada proceso tendrá su propia zona de memoria.

ProcessBuilder

Clase ProcessBuilder:

La clase ProcessBuilder define un par de constructores:

```
ProcessBuilder(String... command)

ProcessBuilder(List<String> command)
```

```
String command1 = "notepad.exe prueba1.txt"
ProcessBuilder pb = new ProcessBuilder(command1);
Process hijo = pb.start();
// 2ª forma: usando un array de cadenas. Funciona con parámetros
String[] command2 = {"cmd", "/c", "dir", "/o"};
ProcessBuilder pb = new ProcessBuilder(command2);
```

ProcessBuilder

Clase ProcessBuilder:

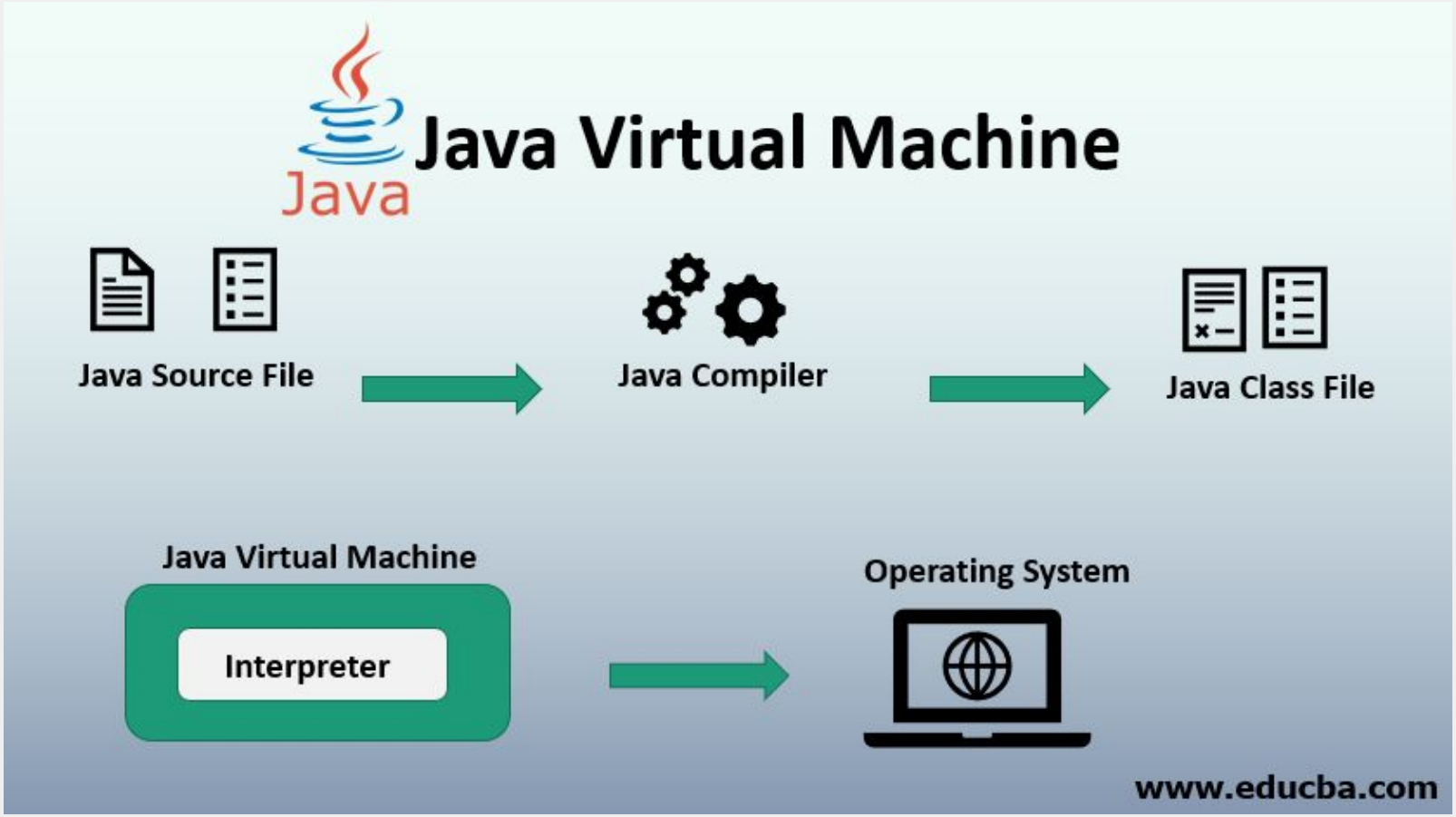
- **Cambiar el directorio** de trabajo por defecto llamando al método directory y pasándole un objeto de tipo File. Por defecto, el directorio de trabajo se establece al valor de la variable del sistema user.dir

```
// Cambia el directorio de trabajo a la carpeta personal del usuario
pb.directory(new File(System.getProperty("user.home")));
```

- Modificar las variables de entorno:

```
Map<String, String> environment = pbuilder.environment();
// Agrego un nuevo directorio a la ruta del sistema
String systemPath = environment.get("path") + ";c:/users/public";
environment.replace("path", systemPath);
// Agrego variable de entorno BIENVENIDA
environment.put("BIENVENIDA", "Hola Mundo");
```

Procesos en Java



Ejecutables y lanzadores

Un **ejecutable** es un archivo con la estructura necesaria para que el sistema operativo pueda poner en marcha el programa que hay dentro. En Windows, los ejecutables suelen ser archivos con la extension **.EXE**.

Java genera ficheros .JAR o .CLASS. Estos ficheros no son ejecutables sino que son archivos que el **intérprete de JAVA** (el archivo java.exe) leerá y ejecutará.

Un archivo .CLASS puede desensamblarse utilizando el comando javap -c <archivo.class> .

Ejecutables y lanzadores: Desde una clase Java

Objetivo: Ahora vamos a ejecutar una clase, que tiene un main, desde otra clase.

Si se trata de una clase Java: primero tenemos que compilar y luego ejecutar. Así que nuestro primer lanzador va a ser para ejecutar el comando que compile nuestra clase.

Usaremos el comando javac.

javac -d rutaDirectorioDondeGeneraClass rutaYNombreFichero.java

Si además mi clase utiliza otras clases tendré que añadir el parámetro -sourcepath para indicar la ruta a estos ficheros.

Ejecutables y lanzadores

Lanzador que **compila** una clase llamada ProcesoHijo.java:

```
public class LanzaComandosJava {
    private final static String directorioGenerarClasses = "target\\classes";
    private final static String rutaSourceJava = "src\\main\\java\\unidad1\\ProcesoHijo.java";

    public int lanzaCompilador(String rutaFicheroFuente, String rutaParaGenerarClass) {
        ProcessBuilder pb;
        try {
            String[] comando = { "javac", "-d", directorioGenerarClasses, rutaSourceJava};
            pb = new ProcessBuilder(comando);
            pb.inheritIO(); // La salidas y los errores se ven por consola
            Process proceso = pb.start();
            int exit = proceso.waitFor(); //esperamos a que javac termine
l           logger.debug("javac terminó con código {}", exit);

        } catch (Exception e) {          logger.error(e.getMessage());          }
        return exit;
    }
    public static void main(String[] args) {
        LanzaComandosJava l = new LanzaComandosJava();
        int codigo= l.lanzaCompilador(rutaSourceJava, directorioGenerarClasses);
    } }
```

Ejecutables y lanzadores

Lanzador que **ejecuta** una clase:

```
private final static String directorioGenerarClasses = "target\\classes";
public int ejecutaClase (String clase) { //Sin la extensión
    int resultado = -1;
    String claseCompleta =  clase;
   try {
            String[] comando = {"java", "-cp",
             directorioGenerarClasses,
            claseCompleta, “hola”, “holita” };
            pb = new ProcessBuilder(comando);
            pb.redirectErrorStream(true);
            pb.inheritIO();
            Process proceso = pb.start();
            int exit = proceso.waitFor();
            System.out.println(exit);
        } catch (Exception e) {
//OjO estoy capturando Exception, para que quepa. Habría que capturar diferentes excepciones
            System.out.println(e.getMessage());
        }}
```

Ejecutables y lanzadores

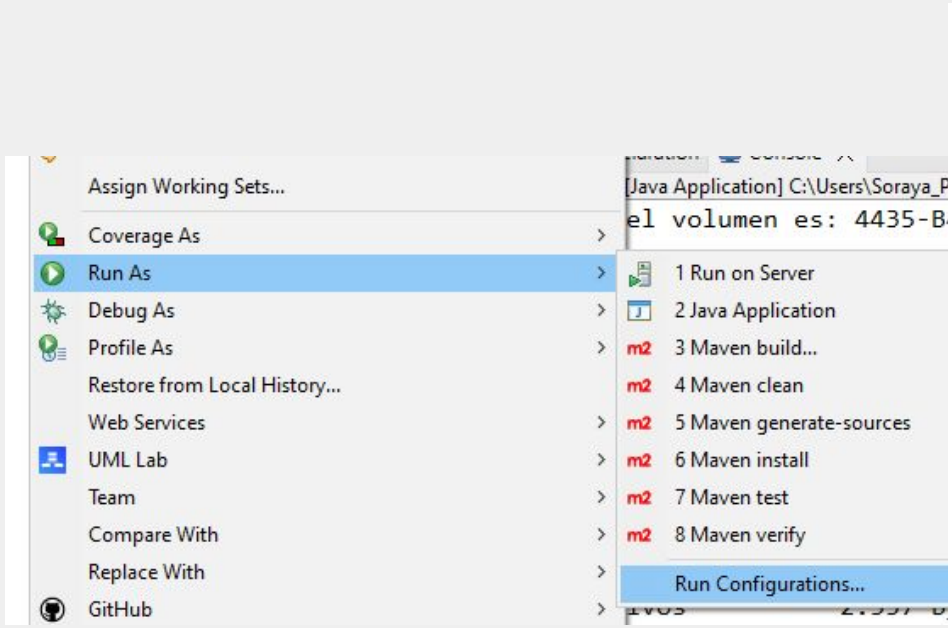
Es tu turno....

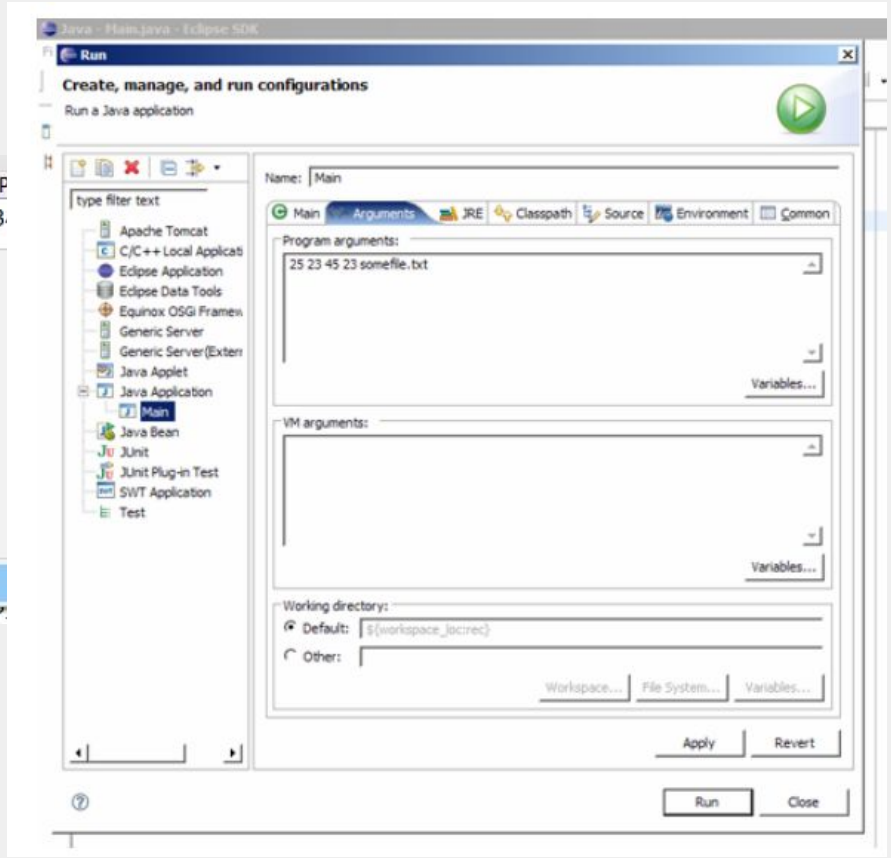
1. El código anterior trabaja con rutas Windows. Modifica el programa anterior para hacerlo portable. Path.of(...) o File.separator DIR_CLASSES = Path.of("target", "classes");
2. Añade comprobación de que existe el directorio funete
3. Ahora:
   a. Genera una clase llamada Hija.java que tenga un main que imprima en el log.
   b. Genera un nuevo método en la clase anterior que se llama ejecutaClase que ejecute el .class de la clase anterior
   c. En tu método main: llama a tu método para que compile la clase y posteriormente llama al método que acabas de hacer para que se ejecute.

Ejecutables y lanzadores: Desde el propio IDE

Objetivo: llamar a un método main de una clase pasándole argumentos desde Eclipse. Para ejecutar este programa desde dentro de Eclipse es necesario indicar que deseamos enviar argumentos al programa.

Introduce los argumentos separados por espacios





Ejecutables y lanzadores

Es tu turno....

2. Ahora:
   a. Genera una clase llamada Hija.java que tenga un main que **imprima en el log**.
   b. Genera un n**uevo método** en la clase anterior que se llama **ejecutaClase que ejecute el .class** de la clase anterior
   c. En tu método **main**: llama a tu método para que **compile** la clase y posteriormente llama al método que acabas de hacer para que se **ejecute**.

Información y comunicación entre procesos

Una vez que un proceso hijo **está en ejecución** podemos obtener información acerca de ese proceso a través de la clase *java.lang.ProcessHandle.Info:*. Podemos saber:

- *El* comando usado para lanzar el proceso
- Los argumentos/parámetros que recibió el proceso
- El instante de tiempo en el que se inició el proceso
- El tiempo de CPU que ha usado el proceso
- El usuario que lo ha lanzado

Información y comunicación entre procesos

java.lang.ProcessHandle.Info:.

*Ejemplo de código de la clase **Hija** que obtiene información del proceso en curso:*

```
public static void main(String[] args) {

    ProcessHandle processHandle = ProcessHandle.current();
    ProcessHandle.Info processInfo = processHandle.info();
    System.out.println("PID: " + processHandle.pid());
    System.out.println("Argumentos: " + processInfo.arguments());
    System.out.println("Commandos: " + processInfo.command());
    System.out.println("Inicio: " + processInfo.startInstant());
    System.out.println("Duración CPU: " + processInfo.totalCpuDuration());
    System.out.println("Usuario: " + processInfo.user());
}
```

Información y comunicación entre procesos

java.lang.ProcessHandle.Info:.

De igual manera, en la clase Padre se podría obtener información del proceso lanzado:

```
Process hijo = processBuilder.start();
ProcessHandle manejadorHijo = hijo.toHandle();
ProcessHandle.Info informacionHijo = manejadorHijo.info();
```

Comunicación entre procesos

En Java, por defecto, la entrada y salida estándar del hijo se conectan mediante tuberías al padre, no a la consola de **E/S directamente**.

Importante: Lo comentado en el punto anterior, no cambia nada la manera de programar el proceso hijo. Los procesos hijos deben ser programados sin tener en cuenta quién pueda lanzarlos o si van a ser hijos. Deben poder ser ejecutados de manera independiente.

Comunicación entre procesos: De Padre a Hijo

- **En la clase Padre**

  Añadiremos, junto al nombre del proceso Hijo, los parámetros que deseamos que el proceso Hijo reciba como argumentos. Por ejemplo, en este caso le pasamos la palabra par y e valor 10.

```
String[] comando = {"java", "-cp", "target/classes", rutaSource,"par", "10" };
```

- **En la clase Hijo**

  Haremos la lectura del parámetro del main String[] args

```
args[0]
```

```
args[1]
```

  Ten en cuenta que se recibe una cadena, por lo que tendremos que convertir de String al tipo que necesitemos

Comunicación entre procesos: De Padre a Hijo

Es tu turno:

- Genera una clase llamada **Hija2 que reciba dos argumentos** en su método main. Los lea y los imprima junto con el nombre de la clase
- Ahora modifica el método que **ejecuta la clase** para que reciba estos dos parámetros como argumentos y los agregue en la llamada al comando java.

Comunicación entre procesos: Enviar resultado Hijo a Padre

El hijo no "devuelve" valores. En realidad, escribe en sus flujos estándar y termina con un código de salida.

EL padre los recoge mediante los flujos.

Enviar un resultado de Proceso Hijo a Proceso Padre: getInputStream:

- En el hijo: `System.out.println(valor devuelto);`
- En el padre: `InputStream salidaHijoStream = proceso.getInputStream();`

  Este método lee **el stream de salida del proceso**, es decir, sirve para leer lo que el comando ejecutado (proceso hijo) ha devuelto.

- Puedes devolver diferentes cosas separando por comas

Comunicación entre procesos: Enviar error de Hijo a Padre

El proceso hijo devuelve un **código de salida:**

- **En el proceso Hijo:**
  - Si el proceso termina correctamente el código será 0
  - Si hay un error: `System.exit(codigoErrorDistintoDeCero);`

- **En el proceso Padre:**
  - Creo una tabla de códigos de salida enteros y guardo:

    `exitCode[i] = p.waitFor();` Donde p es el proceso Hijo

Comunicación entre procesos: Enviar resultado Hijo a Padre

PADRE – Padre.java (recibe)

```
// 1. Preparar y lanzar el hijo
ProcessBuilder pb = new ProcessBuilder(
    "java", "-cp", "bin", "Hijo", "datos.txt");
// (opcional) unir stderr a stdout
pb.redirectErrorStream(true);
Process p = pb.start();

// 2. Leer lo que el hijo escribe
A BufferedReader br = new BufferedReader(
    new InputStreamReader( p.getInputStream() ));
String linea;
while ((linea = br.readLine()) != null) {
    // "total:42" -> 42
    int v = Integer.parseInt(linea.split(":")[1].trim());
}

B // errores del hijo (si no se unieron)
p.getErrorStream()

C // 3. Esperar fin y leer código de salida
int codigo = p.waitFor();     // 0 = OK

D // padre -> hijo (sentido contrario)
p.getOutputStream()
```

A — **Salida estándar (stdout)**
canal principal de datos
`println()` → `getInputStream()`

B — **Salida de error (stderr)**
mensajes de fallo y avisos
`System.err` → `getErrorStream()`

C — **Código de salida**
un entero: éxito o tipo de error
`System.exit(n)` → `waitFor()`

D — Entrada estándar (stdin)
solo si el padre envía datos al hijo
`getOutputStream()` → `System.in`

HIJO – Hijo.java (envía)

```
public static void main(String[] args) {

  // lee el argumento recibido
  String fichero = args[0];

  D // (si hace falta) leer del padre
  new Scanner(System.in)

  int total = contar(fichero);

  A // resultado al padre
  System.out.println(
      "total:" + total);

  B // si algo falla
  System.err.println(
      "No existe " + fichero);

  C // fin: 0 = OK, otro = error
  System.exit(0);
}
```

Ojo con el nombre: lo que el hijo **escribe** (su *output*) el padre lo **lee** con `getInputStream()`. Los nombres se ponen desde el punto de vista del padre.

1. Acordad un formato

Padre e hijo deben pactar el formato de cada línea, por ejemplo `clave:valor`. Valida con `try/catch` de `NumberFormatException`.

2. Lee antes de waitFor()

Si el hijo escribe mucho y nadie lee, el buffer se llena, el hijo se bloquea y `waitFor()` no termina nunca.

3. No uses inheritIO()

`inheritIO()` manda la salida del hijo directamente a la consola. `getInputStream()` queda vacío y el padre no recibe nada.

4. Cierra los flujos

Usa `try-with-resources` o cierra en `finally`. Comprueba siempre el código de salida antes de dar por bueno el resultado.

Comunicación entre procesos: Enviar error de Hijo a Padre

¿Y si queremos propagar la excepción de Hijo a Padre?

Los subprocesos lanzados con ProcessBuilder son procesos independientes, por lo que Java no puede “recoger” una excepción directamente del subproceso

Una excepción no puede pasar de un proceso a otro: cada proceso tiene su propia JVM y su propia memoria. Lo que se hace es convertir la excepción en datos (texto en stderr y un código de salida) y volver a crearla en el padre.

Comunicación entre procesos: Enviar error de Hijo a Padre

¿Y si queremos propagar la excepción de Hijo a Padre?

Opción 1:

La forma más directa es no capturarla completamente en el subproceso y dejar que se lance la excepción. Esta excepción se pintará en la consola de salida (a título informativo)

Para ello, hay que habilitar la comunicación entre proceso y subprocesos:

```
pb.redirectErrorStream(true);
pb.inheritIO();
```

Pero en ese caso perdemos el resto de comunicaciones debido al inherit

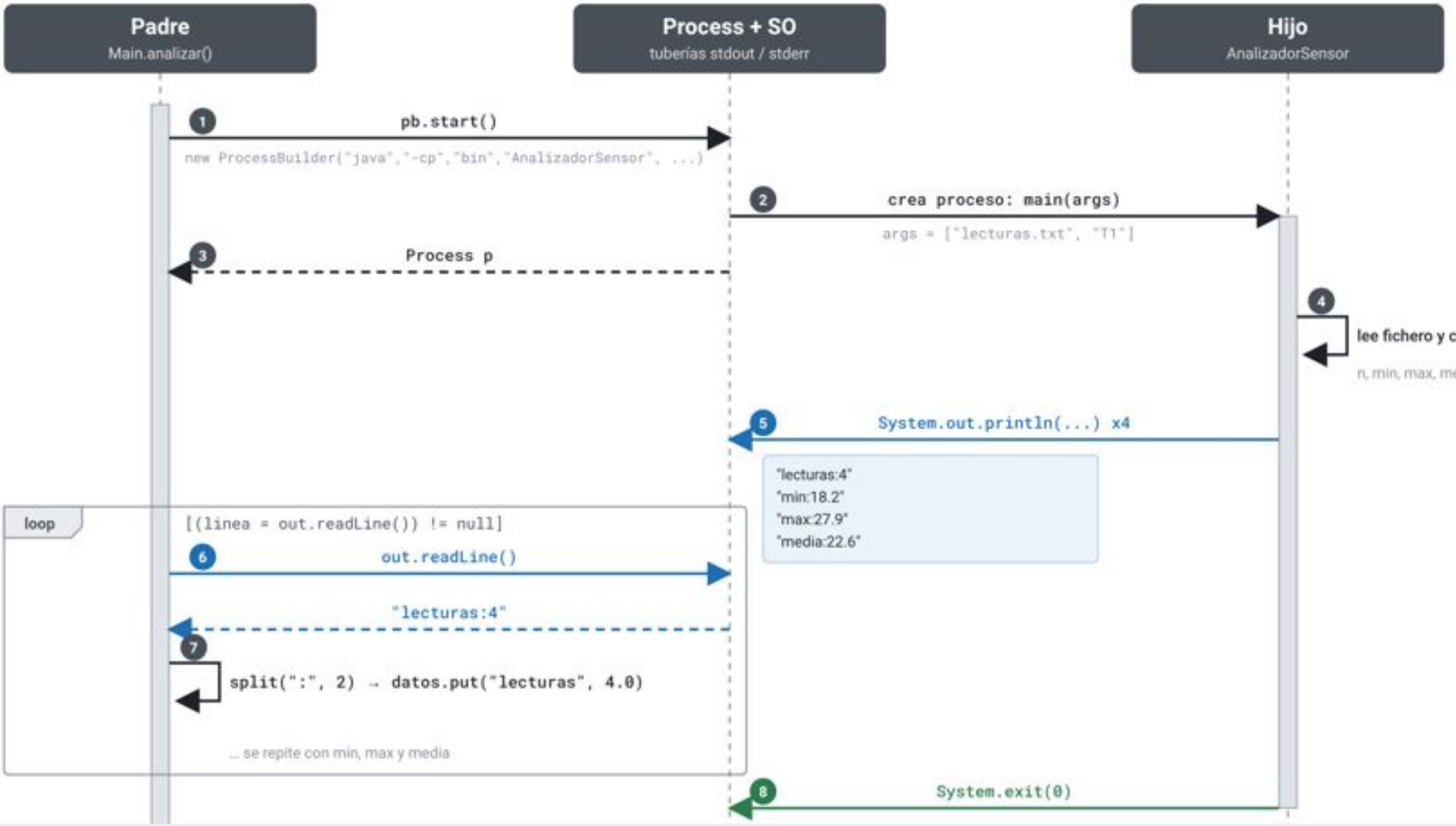
Comunicación entre procesos: Enviar error de Hijo a Padre

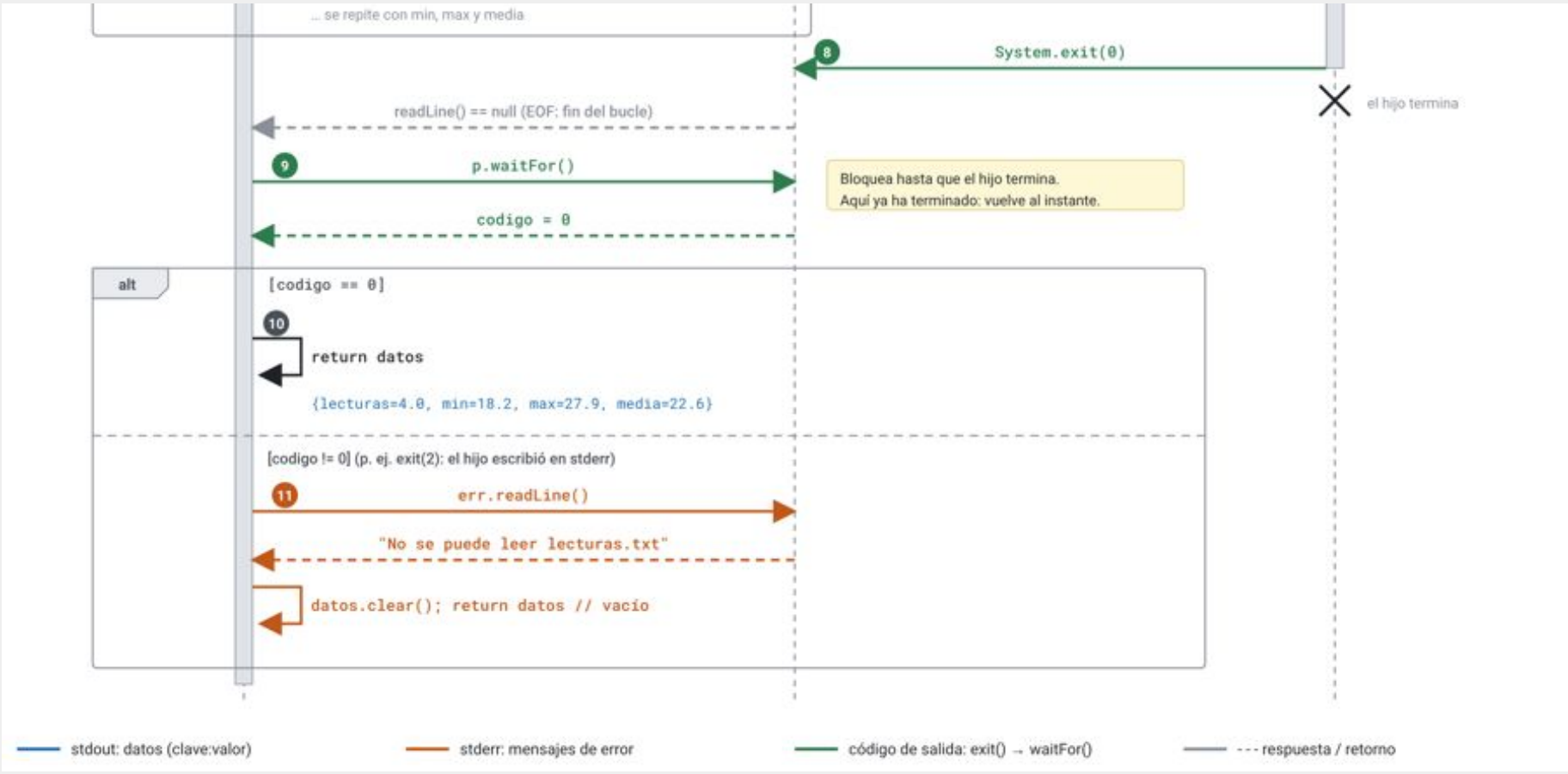
¿Y si queremos propagar la excepción de Hijo a Padre?

Opción 2:

En el proceso Hijo en el catch de cada excepción:

.


Padre
Main.analizar()
Process + SO
tuberías stdout / stderr
Hijo
AnalizadorSensor
1
pb.start()
new ProcessBuilder("java","-cp","bin","AnalizadorSensor", ...)
2
crea proceso: main(args)
args = ["lecturas.txt", "T1"]
3
Process p
4
lee fichero y c
n, min, max, me
5
System.out.println(...) x4
"lecturas:4"
"min:18.2"
"max:27.9"
"media:22.6"
loop
[(linea = out.readLine()) != null]
6
out.readLine()
"lecturas:4"
7
split(":", 2) → datos.put("lecturas", 4.0)
... se repite con min, max y media
8
System.exit(0)


... se repite con min, max y media
8
System.exit(0)
el hijo termina
readLine() == null (EOF: fin del bucle)
9
p.waitFor()
Bloquea hasta que el hijo termina.
Aquí ya ha terminado: vuelve al instante.
codigo = 0
alt
[codigo == 0]
10
return datos
{lecturas=4.0, min=18.2, max=27.9, media=22.6}
[codigo != 0] (p. ej. exit(2): el hijo escribió en stderr)
11
err.readLine()
"No se puede leer lecturas.txt"
datos.clear(); return datos // vacío
stdout: datos (clave:valor)
stderr: mensajes de error
código de salida: exit() → waitFor()
--- respuesta / retorno

Comunicación entre procesos

- pb.redirectErrorStream(**true**); // Redirigir el flujo de error al flujo de salida

  Redirige el flujo de error estándar (stderr) hacia el flujo de salida estándar (stdout). Esto significa que tanto los mensajes de salida normal (stdout) como los mensajes de error (stderr) se fusionarán en el mismo flujo, de modo que ambos serán capturados desde el flujo de salida estándar.

Comunicación entre procesos

- *pb.inheritIO();*

  El método inheritIO() en la clase ProcessBuilder en Java se utiliza para redirigir el flujo de entrada estándar (stdin), salida estándar (stdout) y error estándar (stderr) del proceso que vas a ejecutar hacia los mismos flujos que el proceso padre (es decir, el proceso Java que está ejecutando ese comando). En otras palabras, hace que el proceso hijo herede los flujos de entrada/salida del proceso Java que lo crea. Con inherit las excepciones se ven por consola, se pierde el poder propagar excepciones

  Con INHERIT, el sistema operativo conecta los flujos del hijo directamente a la consola del padre. No se crea ninguna tubería entre los dos procesos, así que no hay nada que el padre pueda leer.

  No recibes son los datos (stdout) ni el detalle del error (stderr) como texto que el padre pueda procesar.

Comunicación entre procesos

- *pb.inheritIO();*

  Si quieres ver los errores en consola y además recibir los datos, no uses inheritIO(). Hereda solo stderr:

  java

  pb.redirectError(ProcessBuilder.Redirect.INHERIT); // errores visibles en consola

  // stdout se queda en PIPE: el padre sigue leyendo los datos

  Y si quieres que el padre trate la excepción y la relance, usa *redirectError(fichero)*, como en el ejemplo anterior.

Comunicación entre procesos: Enviar error de Hijo a Padre

¿Y si queremos propagar la excepción de Hijo a Padre?

- **En el proceso Hijo:**

  Tan sólo tenemos que lanzar y propagar la excepción correspondiente:

  - Lanzamos la excepción en el proceso hijo:

```
throw new FileNotFoundException("Fichero no encontrado: ", rutaYNombreFichero);
```

  - Y el método main del Hijo propagará la excepción al padre:

```
public static void main(String[] args) throws FileNotFoundException{
```

Comunicación entre procesos: Enviar error de Hijo a Padre

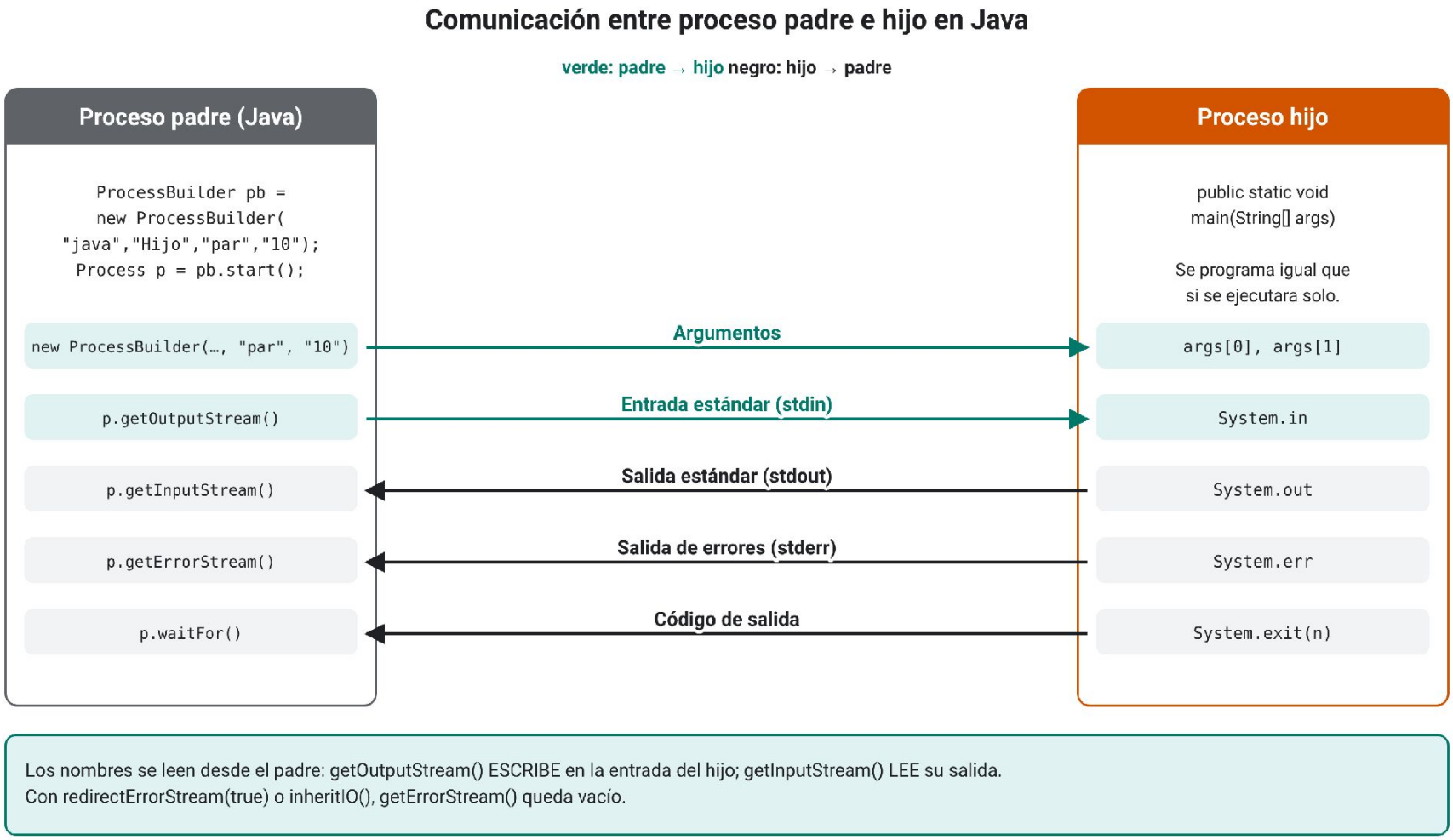
¿Y si queremos propagar la excepción de Hijo a Padre?

- **En el proceso Padre:**

  En este caso, es el padre el que tiene que leer si ha ocurrido un error. En ese caso usamos el método getErrorStream() de ProcessBuilder

  InputStream getErrorStream()

Información y comunicación entre procesos



Los nombres se leen desde el padre: getOutputStream() ESCRIBE en la entrada del hijo; getInputStream() LEE su salida.
Con redirectErrorStream(true) o inheritIO(), getErrorStream() queda vacío.

Comunicación entre procesos: Enviar error de Hijo a Padre

¿Y si queremos propagar la excepción de Hijo a Padre?

- **En el proceso Padre:**

```
BufferedReader errorBuffer =
                        new BufferedReader(new InputStreamReader(p.getErrorStream()));
  // Leer errorBuffer y lanzar excepción en el padre si hay algo
    String lineaError = errorBuffer .readLine()  != null ? errorBuffer .readLine()  :"";
  StringBuilder errMsg = new StringBuilder();
   while (lineaError!= null) {
        errMsg.append(lineaError ).append("\n");
       //Va concatenando todo lo que hay en  el errorBuffer
       lineaError = errorBuffer .readLine() ;
       }
  int exitCode = p.waitFor();
  if (exitCode != 0)
{ //Si ha ocurrido un Error, lanzo excepción con la cadena de error
            throw new RuntimeException("Error en subproceso"  + errMsg.toString());
}
```

Anexo: InputStream y BufferedReader

Los métodos p.getInputStream() (para leer la salida del Hijo desde el padre) y p.getErrorStream() (para leer los errores ocurridos en el Hijo desde el padre)

→ Devuelven **InputStream**.

En ambos casos, este InputStream tengo que pasarlo a **BufferedReader**:

```
BufferedReader reader=
        new BufferedReader(new InputStreamReader(hijo.getInputStream()));
BufferedReader stdError =
        new BufferedReader(new InputStreamReader(hijo.getErrorStream()));
```

La lectura de la clase BufferedReader es similar a la que hemos visto para ficheros:

```
        String linea = reader.readLine();
        while ((linea != null)) {
            System.out.println( linea);
            linea = reader.readLine();          }
```

Comunicación entre procesos

Además puedo redirigir lo que devuelven estos comandos a un fichero:

```
processBuilder.redirectOutput(new File("comandosEntrada.txt"));
processBuilder.redirectError(new File("error.txt"));
processBuilder.redirectInput(new File("salida.txt"));
```

Comunicación entre procesos

```
public static void main(String[] args) {
            ComunicacionSalidaErrores comunicacion = new ComunicacionSalidaErrores();
    // Crear un ProcessBuilder con un comando que generará un error
    ProcessBuilder processBuilder = new ProcessBuilder();
    processBuilder.command("cmd",  "cd /noexiste");
    try {
        Process hijo = processBuilder.start();// Iniciar el proceso
        BufferedReader stdOutput = new BufferedReader(new InputStreamReader(hijo.getInputStream()));
        //Leer lo que le llega al proceso hijo
        BufferedReader stdError = new BufferedReader(new InputStreamReader(hijo.getErrorStream()));
        //Leo la salida de errores
        comunicacion.escriboInformacionProcesoEnFicheroLog(stdOutput);
        comunicacion.escriboInformacionProcesoEnFicheroLog(stdError);
        // Esperar a que el proceso termine
        int exitCode = hijo.waitFor();
          comunicacion.escriboInformacionProcesoEnFicheroLog("\nProceso finalizado con código: " + exitCode);
    } catch (IOException | InterruptedException e) {
        e.printStackTrace();
    }
 }
```

Comunicación entre procesos

```
private void escriboInformacionProcesoEnFicheroLog(BufferedReader
buffer) throws IOException
      {

       String line;
             logger.debug("Salida:");
        while ((line = buffer.readLine()) != null) {
            logger.debug(line);
         }
      }
```


¿Dudas?